MODULE 15

Lab 3: Full Feature Sprint

Duration	Format	Audience	Day / Position
60 min	**Hands-On Lab**	**Mixed Engineers**	**Day 2 / Seventh**

LAB OVERVIEW

This is the capstone lab of the course. In 60 minutes, you will build a complete feature end-to-end using every technique from Day 1 and Day 2: decomposition, plan mode, subagents, quality gates, error handling, performance optimization, and PR workflows. The goal is to simulate a real development sprint with AI assistance.

This is a semi-guided sprint. The exercises provide structure and checkpoints, but you make the decisions: which approach, which patterns, which trade-offs. The instructor is available for help but won't demo each step.

- Sprint structure with checkpoints
- You make the decisions
- Instructor available for help

FEATURE BRIEFS

Track A: Sensor Dashboard API	Track B: Automated Report Pipeline
Build a FastAPI-based REST API that serves sensor toolkit data: list sensors, get readings by ID, get validation summary, get statistics.	Build an automated report generator that reads sensor data, validates, cleans, analyzes, and produces a formatted Markdown report.
Endpoints: GET /sensors, GET /sensors/{id}/readings, GET /sensors/{id}/stats, POST /validate	Functions: generate_report(csv_path) → reads CSV, validates, cleans, analyzes, writes report.md
Requires: FastAPI, pydantic models, error handling, OpenAPI docs, tests	Requires: pathlib, datetime formatting, summary statistics, anomaly flagging, Markdown output
Complexity: Medium-High (new dependency, multiple endpoints, integration tests)	Complexity: Medium (new module, file I/O, formatting, unit tests)

SPRINT TIMING

0:00 – 0:08	**Phase 1: Decompose & Plan**

0:08 – 0:15	**Phase 2: Setup & Scaffold**
0:15 – 0:35	**Phase 3: Implement (Core Sprint)**
0:35 – 0:45	**Phase 4: Test & Quality Gate**
0:45 – 0:53	**Phase 5: Refactor & Optimize**
0:53 – 0:60	**Phase 6: PR, Review & Ship**

0:00 – 0:08	PHASE 1: Decompose & Plan

Phase 1: Decompose & Plan (8 min)

Step 1: Create a safety checkpoint

```
cd ~/my-sensor-toolkit
git add -A && git commit -m "pre-sprint: Lab 3 baseline"
git tag v0.2.1-pre-sprint
```

Step 2: Activate plan mode and decompose

```
claude
> Shift+Tab  # Plan mode ON

# Track A:
> [PLAN MODE] I need to add a FastAPI REST API to the
> sensor toolkit. Decompose into implementable steps:
> - List all new files and modifications needed
> - Order steps by dependency (what must be built first)
> - Estimate time per step
> - Identify which steps can use existing validators/cleaners
> - Flag any new dependencies (FastAPI, uvicorn, httpx)

# Track B:
> [PLAN MODE] I need to add an automated report generator.
> Decompose into implementable steps:
> - New module: src/sensor_toolkit/reporter.py
> - Functions: generate_report(csv_path) as entry point
> - Sub-functions: load_data, summarize, flag_anomalies,
>   format_markdown
> - Tests for each function
> - Error handling for missing/malformed CSV files
```

Step 3: Review and approve the plan

```
# Review Claude's plan output:
> Save the plan to docs/sprint-plan.md
> Shift+Tab  # Plan mode OFF
```

✅ PHASE 1 CHECKPOINT

- Git baseline commit and tag created
- docs/sprint-plan.md exists with decomposed steps
- Dependencies identified (Track A: FastAPI, uvicorn, httpx)
- Plan is ordered by dependency — you know what to build first

0:08 – 0:15	PHASE 2: Setup & Scaffold

Phase 2: Setup & Scaffold (7 min)

Step 1: Install dependencies (Track A only)

```
# Track A: Install FastAPI ecosystem
> Add fastapi, uvicorn, and httpx (for testing) to
> requirements-dev.txt and install them.

# Track B: No new dependencies needed
```

Step 2: Create the subagent for this sprint

```
# Create a sprint-specific reviewer subagent:
mkdir -p .claude/agents

# Track A: API reviewer
> /agents → Create new agent → Project-level
> A FastAPI code reviewer that checks: endpoint naming,
> HTTP status codes, request/response models, error
> handling, OpenAPI documentation, and test coverage.
> Read-only tools. Output prioritized findings.

# Track B: Report reviewer
> /agents → Create new agent → Project-level
> A report generator reviewer that checks: Markdown
> formatting, statistics accuracy, anomaly detection
> logic, file handling, and error handling.
> Read-only tools.
```

Step 3: Scaffold the new files

```
# Track A: Scaffold API structure
> Scaffold the API module:
> - src/sensor_toolkit/api.py (FastAPI app)
> - src/sensor_toolkit/models.py (Pydantic models)
> - tests/test_api.py (API tests with httpx)
> Create stub functions with docstrings and type hints.
> Do NOT implement yet — just the structure.

# Track B: Scaffold report module
> Scaffold the reporter module:
> - src/sensor_toolkit/reporter.py
> - tests/test_reporter.py
> Create stub functions: generate_report, load_data,
> summarize_readings, flag_anomalies, format_markdown.
> Docstrings and type hints only. No implementation.
```

✅ PHASE 2 CHECKPOINT

- Dependencies installed (Track A)
- Sprint-specific reviewer subagent created
- New files scaffolded with stubs, docstrings, and type hints

- Tests file created with placeholder test functions

0:15 – 0:35	PHASE 3: Implement (Core Sprint)

Phase 3: Implement — Core Sprint (20 min)

This is the main implementation phase. Follow your sprint plan from Phase 1. Implement one function at a time, test as you go. Use the refactor-test-commit cycle from Module 7.

Implementation Order (Track A)

```
# Step 1: Pydantic models (5 min)
> Implement the Pydantic models in models.py:
> SensorReadingResponse, ValidationSummary,
> SensorStats. Base them on our existing dataclasses.

# Step 2: Core endpoints (10 min)
> Implement the API endpoints in api.py:
> 1. GET /sensors → list unique sensor IDs from sample data
> 2. GET /sensors/{id}/readings → return readings for sensor
> 3. GET /sensors/{id}/stats → return statistics
> 4. POST /validate → validate a reading, return result
> Use existing validators and analyzers. Add proper HTTP
> status codes (200, 404, 422) and error handling.

# Step 3: Wire up tests (5 min)
> Implement test_api.py using httpx TestClient.
> Test each endpoint: happy path + error cases.
```

Implementation Order (Track B)

```
# Step 1: Data loading (5 min)
> Implement load_data(csv_path) in reporter.py.
> Use existing CSV reading patterns. Add error handling
> for missing files and malformed data.

# Step 2: Analysis functions (8 min)
> Implement summarize_readings() and flag_anomalies().
> summarize: count, mean/min/max per field, valid %.
> flag_anomalies: readings outside 2 std devs.

# Step 3: Markdown formatter (4 min)
> Implement format_markdown() that takes the summary
> and anomalies and produces a formatted report with
> headers, tables, and timestamp.

# Step 4: Entry point + tests (3 min)
> Wire up generate_report(csv_path, output_path).
> Write tests for each function.
```

☆ **KEY POINT:** Implement one function at a time. After each function: run its tests, check for lint issues, commit. Don't try to implement everything at once. The pattern: implement → test → lint → commit. Repeat.

⚠ **WARNING:** If you get stuck on a function for more than 3 minutes, use /rewind and try a simpler approach. It's better to have 3 working endpoints than 4 broken ones. Scope management is a key skill.

✅ **PHASE 3 CHECKPOINT (at 0:35)**

- Core functions implemented
- Tests written and passing for each function
- Lint clean (ruff check)
- At least 2-3 git commits during implementation

0:35 – 0:45	PHASE 4: Test & Quality Gate

Phase 4: Test & Quality Gate (10 min)

Step 1: Run full test suite

```
# Run all tests with coverage:
> !pytest tests/ -v --cov=src/sensor_toolkit \
>   --cov-report=term-missing --cov-fail-under=70
```

Step 2: Fill coverage gaps

```
# Ask Claude to fill gaps:
> The following lines are not covered by tests:
> [paste uncovered lines from coverage report]
> Generate tests for these lines. Focus on edge cases:
> - Empty input, None values, malformed data
> - Boundary values for sensor ranges
> - Error paths (file not found, invalid CSV)
```

Step 3: Run the quality gate

```
# Full quality gate:
> !ruff check src/ tests/
> !black --check src/ tests/
> !git diff --stat   # Diff size check
> !pytest --cov=src/sensor_toolkit --cov-fail-under=70

# Ask Claude to fix any issues:
> Fix all ruff violations and format with black.

# Optional: complexity check
> !pip install radon && radon cc src/ -a -nc
```

✅ PHASE 4 CHECKPOINT

- All tests pass
- Coverage ≥ 70% (new code ≥ 80%)
- Ruff lint: zero violations
- Black format: clean
- No complexity > C rating

0:45 – 0:53	PHASE 5: Refactor & Optimize

Phase 5: Refactor & Optimize (8 min)

Step 1: Production error handling

```
# Add production patterns from Module 14:
> Review the new code for error handling gaps:
> 1. Are all exceptions specific (not bare except)?
> 2. Is there structured logging with context?
> 3. Do public functions have fail-soft behavior?
> Fix any gaps.
```

Step 2: Performance check

```
# Track A: Check API response times
> Add timing to each endpoint. Target: <100ms response.

# Track B: Check report generation speed
> Time generate_report() with 1000 readings.
> Target: under 2 seconds.
```

Step 3: Documentation

```
# Generate documentation:
> Add docstrings to all public functions in the new modules.
> Update README.md with:
> - New feature description
> - Usage examples
> - API endpoints (Track A) or CLI usage (Track B)
```

✅ PHASE 5 CHECKPOINT

- Production error handling in place
- Performance meets targets
- All public functions have docstrings
- README updated with new feature

0:53 – 0:60	PHASE 6: PR, Review & Ship

Phase 6: PR, Review & Ship (7 min)

Step 1: Generate PR description

```
# Generate professional PR:
> Generate a GitHub PR description for all changes since
> v0.2.1-pre-sprint. Include: title (feat: ...), summary,
> files modified, testing, breaking changes, reviewer
> checklist. Save to docs/sprint-pr.md
```

Step 2: Run subagent review

```
# Use the sprint-specific reviewer:
> Use the [api-reviewer/report-reviewer] subagent to
> review all changes since v0.2.1-pre-sprint.

# Address critical findings:
> Fix the critical issues identified by the reviewer.

# Re-run tests after fixes:
> !pytest tests/ -v
```

Step 3: Final commit and tag

```
# Commit everything:
> !git add -A
> !git commit -m "feat: add sensor dashboard API" # or report
> !git tag v0.3.0 -m "Lab 3: full feature sprint complete"
```

✅ LAB 3 COMPLETE

- docs/sprint-plan.md — decomposition and implementation plan
- Sprint-specific reviewer subagent in .claude/agents/
- Feature fully implemented with production error handling
- All tests passing, coverage ≥ 70%, lint clean
- Performance meets targets
- README updated with new feature docs
- docs/sprint-pr.md — professional PR description
- Subagent review completed, critical issues addressed
- Final commit with v0.3.0 tag

SPRINT RETROSPECTIVE

What took longer than expected? Why?
Which Claude Code technique saved the most time?
What would you do differently in the next sprint?
Which quality gate caught the most issues?
Is your feature production-ready? What's missing?

COURSE SUMMARY

Module	Topic	Key Takeaway
M1	Big Picture & Safety	Vibe coding + TI safety checklist
M2	Prompting Fundamentals	Zero/few-shot, CoT, prompt hygiene
M3	Advanced Patterns	Decomposition, ReAct, CLAUDE.md
M4	CLI & VS Code	Slash commands, checkpoints, permissions
M5	Automation & Scripting	Headless mode, JSON output, pipelines
M6	Lab 1: Scaffold & Tests	Project bootstrap, first tests
M7	Refactor & Debug	Refactor-test-commit, traceback debugging
M8	Quality Assurance	BICEP, fixtures, property/mutation testing
M9	CI/CD & Release	GitHub Actions, failure analysis, hooks
M10	Agents: Foundations	Subagents, YAML config, guardrails
M11	Agents: Applied	Observation, localization, plan mode, MCP
M12	Lab 2: Assistant Loop	Observe → plan → implement → review → ship
M13	Parallel & PR	Background tasks, worktrees, PR automation
M14	Production & Debt	Error handling, performance, tech debt
M15	Lab 3: Feature Sprint	Everything combined in a real sprint